

STAGE 1: SEARCH ENGINE DATA LAYER & BENCHMARK

Author: Pablo Martínez Suárez | Language: C++17 (Clang/GCC -O2) | Course: Big Data — ULPGC

1. Executive Summary & Architecture Overview

This technical report documents the complete implementation, benchmarking, and empirical analysis of the Stage 1 Data Layer for an autonomous search engine written in C++17. The system strictly adheres to the Cross-Language Benchmark Contract (CONTRACT.md), enabling direct, 1:1 comparable evaluations against Java and Python counterparts. The architecture is cleanly decoupled into four core subsystems: (1) Datalake Storage Engine supporting three distinct directory layout topologies with single-pass incremental detection and idempotent recovery; (2) Metadata Datamart utilizing SQLite with transaction-batched WAL ingestion (> 530,000 rows/s); (3) Inverted Index Datamarts comparing Monolithic JSON, Hierarchical Sharded Folders, and a Relational SQLite Index with real process memory profiling (measuring actual resident set size); and (4) Control Layer orchestrating pipeline ingestion via libcurl and automated multi-datamart state synchronization.

2. Datalake Multi-Scale Evaluation (100 / 1,000 / 10,000 Books)

We evaluated three physical partitioning strategies across scales of 100, 1,000, and 10,000 books: Time-based (YYYYMMDD/HH/), Book-based (<id>/), and Batch-based (batch_N/, 500 books/batch). Metrics include initial write throughput, random lookup latency, single-pass incremental detection time, incremental write time, idempotent recovery duration, and directory metadata overhead.

Scale	Structure	Write (s)	Books/s	Lookup (ms)	Incr. Det (s)	Incr. Wrt (s)	Recovery (s)	Dirs
100	time_based	0.098	1,015.5	0.012	0.0002	0.009	0.019	2
100	book_based	0.117	858.2	0.009	0.0003	0.012	0.014	100
100	batch_based	0.113	881.6	0.011	0.0003	0.010	0.011	1
1,000	time_based	1.130	885.0	0.015	0.0014	0.131	0.171	2
1,000	book_based	1.274	785.2	0.012	0.0039	0.114	0.125	1,000
1,000	batch_based	1.123	890.4	0.013	0.0033	0.112	0.123	2
10,000	time_based	11.672	856.8	0.018	0.0123	1.184	3.996	2
10,000	book_based	12.370	808.4	0.013	0.0400	1.153	8.241	10,000
10,000	batch_based	11.468	872.0	0.015	0.0324	1.050	4.366	20

Datalake Analysis & Findings:

- Write Throughput & Inode Overhead:** time_based and batch_based achieve the highest throughput (~830–1,015 books/s) by minimizing directory creations. book_based creates 10,000 directories at scale, introducing significant filesystem inode table pressure.
- Incremental Processing & Recovery:** Single-pass scan (detectNewBooks) takes only 11.4 ms for 10,000 books in time_based, versus 41.3 ms in book_based due to directory tree recursion overhead. Recovery idempotence verified: 100% of missing files were restored with zero duplicate corruption.

3. Metadata SQLite Datamart Benchmark (Team Extension)

Project Gutenberg headers were parsed via C++ std::regex into structured fields (book_id, title, author, language, ingested_at, header_path, body_path) and inserted into SQLite using explicit transaction chunking (BEGIN TRANSACTION / COMMIT) with prepared statements. Lookups were tested for exact PK (find_by_id) and partial match (find_by_author) using Charles Dickens (repeated) and Lewis Carroll (unique).

Scale (Books)	Insert (s)	Throughput (rows/s)	find_by_id (ms)	Dickens (ms)	Carroll (ms)	DB Size (KB)
100	0.0089	11,205.4	0.230	0.231	0.188	28.0
1,000	0.0038	260,827.1	0.146	0.278	0.242	168.0
10,000	0.0186	536,940.4	0.141	1.454	1.101	1,564.0

Metadata Findings: Transactional batching delivers outstanding write scalability, climbing to 536,940 rows/sec at 10,000 books. Primary key index lookups remain constant at ~0.14 ms (O(1)/O(log N) B-Tree), while unindexed author substring scans scale linearly with dataset cardinality.

4. Inverted Index Datamart Multi-Scale Evaluation & Memory Profiling

Following the shared contract, we indexed the corpus using the exact tokenization rule (lowercase ASCII [A-Za-z]+) and evaluated three architectures: 1. Monolithic JSON (single file containing all vocabulary postings loaded into RAM), 2. Hierarchical Sharded (directory-partitioned term files A-Z/_), and 3. Relational SQLite Index (CREATE TABLE inverted_index(term TEXT, book_id INTEGER, PRIMARY KEY (term, book_id))). Each architecture is evaluated under strict worker process isolation to eliminate cross-structure heap contamination, with peak RAM measured empirically via identical resident set size profiling (mach task_info / getrusage).

Scale	Structure	Build (s)	Lookup avg (ms)	Update (ms)	Peak RAM	Disk (MB)
100	json_monolithic	0.051	0.001	137.8	47.5 MB	8.66
100	hierarchical	2.724	0.130	0.4	8.2 MB	4.67
100	sqlite_index	0.742	0.118	0.7	10.1 MB	40.13
1,000	json_monolithic	0.389	0.002	1,038.6	333.3 MB	80.69
1,000	hierarchical	2.939	0.100	0.7	39.9 MB	46.69
1,000	sqlite_index	7.204	0.718	0.7	42.8 MB	406.79
10,000	json_monolithic	5.081	0.006	11,914.7	1307.3 MB	801.04
10,000	hierarchical	6.437	0.458	3.9	344.4 MB	466.89
10,000	sqlite_index	80.208	4.443	2.0	347.4 MB	4,225.39

Inverted Index Architectural Trade-offs & Memory Mechanics:

- **Monolithic JSON:** Delivers instantaneous query lookup (< 0.007 ms) once loaded into RAM, with resident set size scaling directly with corpus postings (47.5 MB → 333.3 MB → 1,307.3 MB). However, it suffers fatal incremental update degradation (11.9 seconds per book at 10,000 scale) due to complete file rewriting.
- **Hierarchical Index:** Streaming file operations provide localized atomic updates (3.9 ms per book) without global file locks. OS filesystem inode caches and directory stream buffers result in clean, monotonic resident memory scaling (8.2 MB → 39.9 MB → 344.4 MB) while maintaining a compact disk footprint (466.9 MB).
- **SQLite Relational Index:** Guarantees full ACID transactional integrity with stable, monotonic resident memory footprint (10.1 MB → 42.8 MB → 347.4 MB) under process isolation. Incremental updates remain exceptionally fast and constant (2.0 ms) even at 10,000 books, trading off higher disk footprint (4,225 MB) for indexed relational access.

5. Automated Testing Suite & Contract Compliance

The C++ module includes a comprehensive automated unit test suite integrated into CMake (ctest) and standalone binary execution. 16 distinct test suites with 119 individual assertions verify: (1) Tokenizer regex & ASCII compliance; (2) Datalake path generation, multi-layout saving, single-pass incremental change detection, and recovery idempotence; (3) Metadata extraction and SQLite CRUD transactions; (4) Inverted Index construction, search correctness, and cross-structure consistency (identical results across JSON, Hierarchical, and SQLite); (5) QueryEngine Boolean AND/OR operations; and (6) Control Layer state machine transitions. **Current Test Results: 16 passed, 0 failed (100% pass rate).**

6. Cross-Language Conclusions & Summary

The C++ implementation demonstrates near-zero runtime overhead, exceptional raw I/O throughput, and sub-millisecond query execution. By aligning tokenization rules, query workloads, and synthetic scaling factors via CONTRACT.md, the C++ results establish an exact empirical baseline for comparing Java and Python in the final cross-language benchmark evaluation. All benchmark metrics, JSON datamarts, and test logs are fully version-controlled in the repository under c++/datamarts/ for verifiable reproducibility.